Neural Networks — Key Concepts

Lesson 9 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

4 December 2026 · a one-page index

Contents

One neuron 1
A hidden layer 1
How a network learns 2
Networks in practice 2

An index, not a summary. Each entry says what a term means and where the handout treats it properly. Use this to find your way back into the text, or to check before an exercise that there is no word here you could not define.

One neuron

What a network adds. Earlier classifiers fixed the shape of the boundary before training; a network learns the shape itself. → § 1

Unit, or neuron. A weighted sum of the inputs plus a bias, passed through an activation. Lesson 4's logistic regression is one unit. → § 2.1

Perceptron. The 1958 ancestor: the same weighted sum, with a hard step in place of the sigmoid. → § 2.1

A neuron is a line. The points a unit is undecided about, where its weighted sum is zero, form a straight line; one unit only says which side of it a point is on. → § 2.1

The ceiling. When the recorded labels themselves contain mistakes, no model can score above one minus the mistake rate. → § 1.1

Why one line fails on a disc. Whatever a line gains on one side of a centred region it gives back on the other, so the best fit barely tilts. → § 2.3

A hidden layer

Hidden layer. Units between the inputs and the output, whose values nobody labels. → § 3.2

Rectified linear unit (ReLU). The activation max(0, z): zero for negative z, z itself otherwise. → § 3.2

Exclusive-or (XOR). True when exactly one of two conditions holds; the smallest problem no single line can separate. → § 3.1

Moving the data. A hidden layer computes new coordinates in which a single line separates the classes. → § 3.2

Representation learning. Learning those new coordinates from data, rather than designing them by hand. → § 3.2

Forward pass. Computing the output from the input, layer by layer: weighted sums, then activations. → § 3.3

Multilayer perceptron (MLP). A network of fully connected layers; scikit-learn's `MLPClassifier` trains one with the hidden sizes you choose. → § 3.4

Each hidden unit is a line. Its weights are one column of the first weight matrix; the output unit votes over which side of each line a point lies on. → § 3.4

Width and findability. A layer just wide enough to solve a problem is often not found by training from a random start; a wider one is found far more often. → § 3.4

How a network learns

The training loop. Predict a mini-batch, measure the loss, send the blame back, move every weight against its blame, repeat. → § 4.1

Mini-batch. A small random group of training examples used for one step of training. → § 4.1

Epoch. One pass through all the mini-batches, so through every training example once. → § 4.1

Cross-entropy loss. Lesson 4's log-loss: small when the prediction is close to the truth, large when a confident prediction is wrong. → § 4.2

Gradient step. Each weight moves against the slope of the loss with respect to it, scaled by the learning rate. → § 4.3

The blame on one unit. For a sigmoid unit with cross-entropy, prediction minus truth; each weight's share is that times the input it carried. → § 4.3

Backpropagation. The chain rule applied layer by layer from the output back, giving every weight its blame in one backward sweep. → § 4.4

Learning rate. The step size. Too small and training is still going when it stops; too large and it never settles. → § 4.5

Networks in practice

Keras. The library used for networks in lessons 9 and 10: a list of layers, a loss and an optimiser, then `fit`. → § 5.2

Dense layer. A layer in which every unit sees every input. → § 5.2

Adam. A refined gradient descent that adapts the step for each weight. → § 5.2

Softmax. Turns one score per class into probabilities that add up to one; with two classes it is the sigmoid. → § 5.3

Confidence is not correctness. Softmax says how sure a network is, not how often it is right when that sure. → § 5.3

When a hidden layer pays. When inputs act together; when they act independently a linear model gets most of the way. Fit the linear model first. → § 5.5

Memorising. A network with far more weights than examples can store its training answers; the validation loss shows it before the accuracy does. → § 6.1

Early stopping. Stop when the validation loss has not improved for a while, and keep the best weights seen. → § 6.2

Dropout. Switch off a random fraction of hidden units on every training mini-batch, so no unit relies on another. → § 6.2

Vanishing gradient. The blame shrinks at every layer it passes back through; with sigmoids, whose slope is at most a quarter, a deep stack never learns. → § 6.3

Shuffled pixels. A dense network ignores which pixel is next to which, so a consistent shuffle changes nothing it can see. → § 6.4